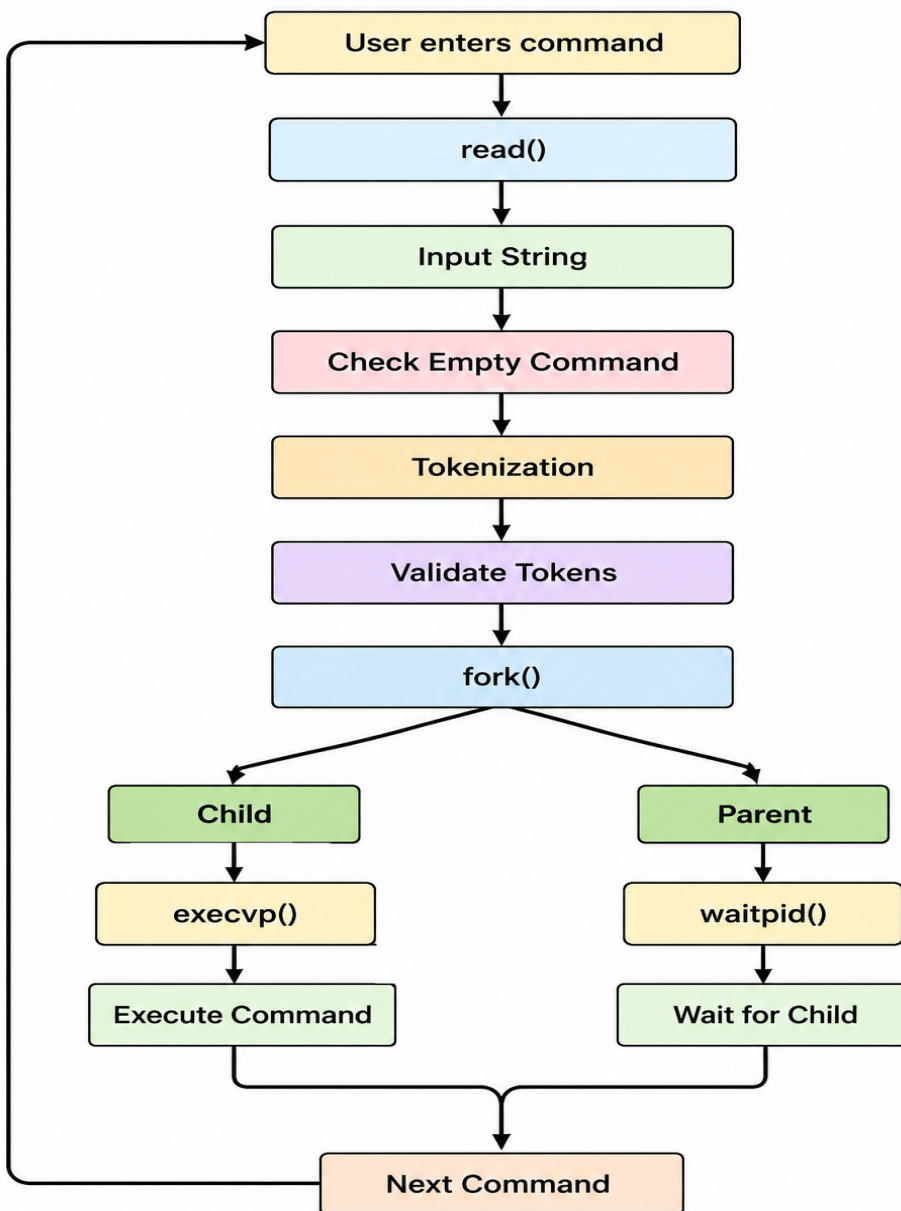


Refer the following link to understand the Parsing and Lexical analysis
<https://www.geeksforgeeks.org/compiler-design/introduction-of-lexical-analysis/>

Task 1:

To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output and Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures

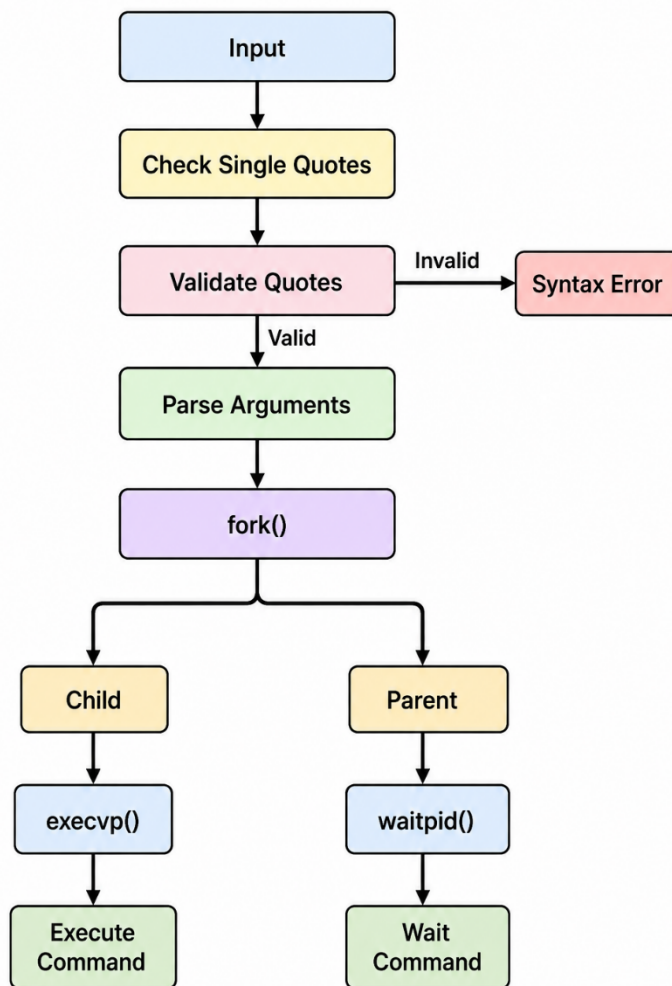
Flow Chart:



Task 2:

To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

Flow Chart:



Example Output:

Case1:

myShell> echo Hello

Parsing Result:

Argument 1: echo

Argument 2: Hello

Child PID: 3210

Hello

Command execution completed.

Case2:

myShell> echo 'Hello World

Syntax Error: Unmatched single quote.

Task 3:

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

Example Output:

Case1:

myShell> echo "Hello \$USER"

--- Parsed Arguments ---

Argument 1: echo

Argument 2: Hello raghu

Child Process

PID = 3022

Hello raghu

Child process completed.

Case2:

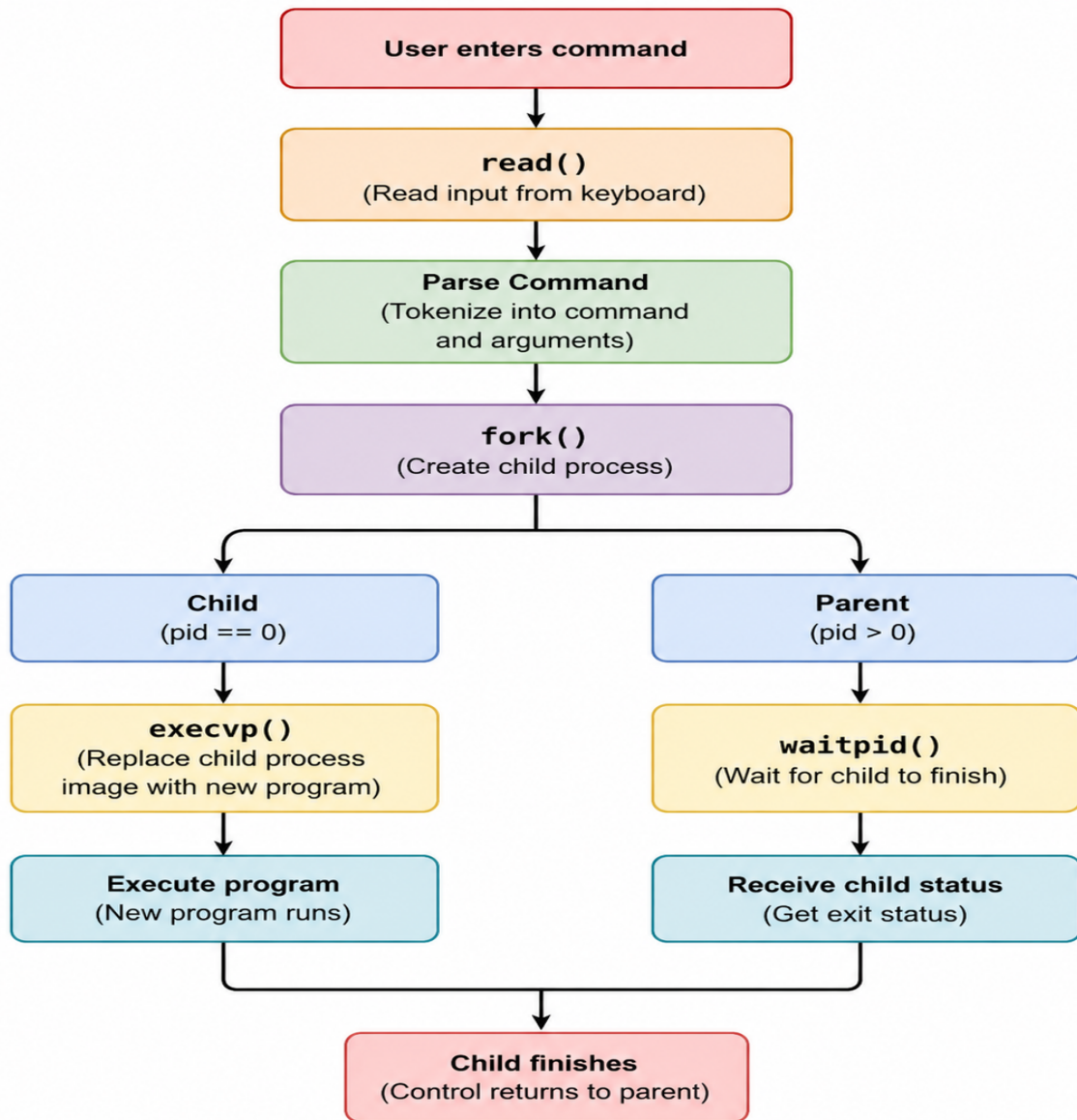
myShell> echo "Hello World

Syntax Error: Unmatched double quote.

Task 4: To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

Task 5: To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.

Flow Chart:



Sample Output:

Case 1:

myshell\$ ls

Parent Process PID: 1200

Created Child PID: 1201

Child Process PID: 1201

file1.txt

file2.txt

Child exited with status: 0

Case 2:

myshell\$ ls @l

Parent Process PID: 82

Child Process PID: 83

Created Child PID: 83

ls: cannot access '@l': No such file or directory

Child exited with status: 2